

EVENTFORCE MANAGEMENT SYSTEM

Salesforce Implementation Project Documentation

Team Leader: Sribalamurugan S

Team Members: Prasanna M, Moosa T, Vignesh R, Samyganesh K

College: Arasu Engineering College

Registered Mail ID: Sribalamurgan9788@gmail.com

Team ID: SWTID-2026-9692

Platform: Salesforce Developer Edition Org (Lightning Experience)

Submission Date: 03.10.2026

Contents

1. Project Overview
 2. Objectives
 3. Problem Statement and Requirements
 4. Project Planning
 5. Implementation (Data Model, Validation, Formula, Flow, Apex, Approval, App and Reports, Security, Data Migration)
 6. Testing
 7. Challenges Faced and Fixes
 8. Limitations and Future Scope
 9. Conclusion
- Appendix A: Apex Source Code

1. Project Overview

The EventForce Management System is a Salesforce CRM application for an event planning business. In many event offices, client bookings, venue reservations, vendor coordination, reminders and cancellations are handled through separate manual steps, which makes them slow and error-prone. This project brings events, clients, venues, vendors, feedback, approvals and reporting into one Salesforce org so that coordinators, vendor managers and administrators work from one shared and current set of records.

The application is built on five custom objects, Event, Client, Vendor, Venue and Feedback, together with the EventVendor junction object that links events and vendors in a many-to-many relationship. A formula field calculates the event budget, a validation rule checks client email addresses, and a record-triggered Flow sends the client a reminder three days before a confirmed event. Apex Triggers and a helper class prevent double booking and keep venue availability current, while a Batch and Scheduled Apex job marks past events as Completed every night. Event cancellations go through an Approval Process. A Lightning App, a report and a dashboard give the team a view of event operations, and access is controlled with Profiles, Roles, a Permission Set, Organization-Wide Defaults and a sharing rule.

All configuration and code was created in a Salesforce Developer Edition org, working in Lightning Experience.

2. Objectives

- Hold client, event, venue, vendor and feedback data in one structured system.
- Streamline event booking and venue reservation.
- Prevent double booking of venues automatically with Apex.
- Support many-to-many assignment of vendors to events.
- Send clients an automatic reminder before a confirmed event.
- Route event cancellation requests through a controlled approval process.
- Keep client feedback linked to both the event and the client.
- Give management reports and a dashboard for event operations.
- Limit what each kind of user can see and do using Profiles, Roles, Permission Sets, OWD and sharing rules.

3. Problem Statement and Requirements

3.1 Problem Statement

Event planning involves clients, venues, vendors and dates at the same time. Without a central system, client and event details are kept in several places, venues can be assigned to more than one event, vendors are tracked separately, reminders are sent by hand, cancellations have no formal approval, and nobody has one complete view of operations. The result is slow coordination, inconsistent data and an unreliable client experience. Eight customer problem statements (PS-1 to PS-8) were captured during ideation from the Event Operations Manager, Event Coordinator, Venue Manager, Vendor Manager and Event Administrator roles.

PS	Role	Problem
PS-1	Event Operations Manager	Client, event, venue and vendor information is difficult to manage centrally.
PS-2	Event Coordinator	Event details, client information and venue reservations need multiple manual activities.
PS-3	Venue Manager	The same venue may be assigned to multiple events.
PS-4	Vendor Manager	Vendor details and event assignments are managed separately.
PS-5	Event Coordinator	Reminders are handled manually.
PS-6	Event Administrator	Cancellation requests need approval and status updates.
PS-7	Event Coordinator	Feedback must be kept with event and client details.
PS-8	Event Operations Manager	Operational information is not visible in one place.

3.2 Users and Their Needs

User	What they need from the system
Event Coordinator	To create events, link clients, venues and vendors, and send reminders.
Vendor Manager	To manage vendors and see the events they are assigned to.
Venue Manager	To keep venue availability accurate and avoid double booking.
Event Administrator	To configure objects, flows, security and monitor the automation.
Event Operations Manager	To see reports and dashboards of event activity.
Client	To get timely event updates and give feedback.

3.3 Functional Requirements

ID	Requirement	Delivered by
FR-01	Event management	Event__c with Event Type, Date, Status and Budget
FR-02	Client management	Client__c, email validation rule
FR-03	Venue management	Venue__c with Availability Status
FR-04	Vendor management	Vendor__c with Service Type and Status
FR-05	Event-vendor assignment	EventVendor__c junction object
FR-06	Venue availability and double-booking prevention	PreventDoubleBooking trigger, VenueStatusHelper
FR-07	Automated event reminders	Record-triggered Flow with scheduled path
FR-08	Event cancellation approval	Event_Cancellation_Process approval process
FR-09	Feedback management	Feedback__c with lookups to Event and Client
FR-10	Reports and dashboards	Upcoming Events by Month report, EventForce Operations Dashboard
FR-11	Event status automation	Approval field updates, VenueStatusHelper, nightly Batch Apex
FR-12	Data migration	Data Import Wizard

3.4 Non-Functional Requirements

- Usability: a Lightning App for easy event management and navigation.
- Security: access by profile, role and permission.
- Reliability: accurate automation, approvals and venue booking validation.
- Performance: efficient processing of event and operational data.
- Availability: cloud-based Salesforce access for authorized users.
- Scalability: room for more events, clients, vendors, venues and users.
- Maintainability: Flows, Apex, validation rules and security that can be monitored and updated.
- Data integrity: validation and controlled data migration keep records accurate.

3.5 Technology Stack

Layer	Technology used
Platform	Salesforce CRM, Developer Edition, Lightning Experience
Data model	Custom objects Event__c, Client__c, Vendor__c, Venue__c, Feedback__c, EventVendor__c; custom fields, formula field
Declarative automation	Record-Triggered Flow, Validation Rule, Approval Process, Email Templates

Layer	Technology used
Programmatic	Apex Trigger (EventTrigger13, PreventDoubleBooking), Apex classes (VenueStatusHelper), Batch Apex, Scheduled Apex
Interface and analytics	Lightning App (Event Planner), reports and dashboards
Security	Profiles, Roles, Permission Sets, OWD, sharing rules
Tools	Setup, Object Manager, Flow Builder, Developer Console, Data Import Wizard

3.6 How Data Moves Through the System

Step	What happens	Result
1	Coordinator enters client details and event details.	Client and Event records created.
2	Validation checks the client email format.	Invalid emails are rejected.
3	Event is saved with type, date and status.	Event Budget is calculated from the Event Type.
4	A venue is selected and PreventDoubleBooking checks for another event at that venue on the same date.	Double booking is blocked.
5	Vendors are assigned through EventVendor records.	Many-to-many assignment stored.
6	Event Status is set to Confirmed.	VenueStatusHelper sets the venue to Reserved.
7	Three days before Event Date, the Flow emails the client with the owner in CC.	Client reminded.
8	Cancellation: status set to Pending Cancellation and the record is submitted for approval.	Approved sets status Canceled and venue Available; rejected sets status Rejected. Emails sent.
9	Nightly Scheduled Apex runs the batch job.	Past events marked Completed.
10	Client feedback is captured against the event and client.	Feedback record exists.
11	Management opens the report and dashboard.	Upcoming events visible by month.

4. Project Planning

The project followed an Agile approach: epics were broken into user stories, each story was given story points, and the work was delivered in four sprints.

4.1 Product Backlog

Sprint	Epic	Story	User story / task	Pts	Priority
1	Event Management	US-01	Create events with type, date and status so that event information is standardised.	3	High
1	Client and Venue Management	US-02	Associate a client and venue with an event so that coordination is centralised.	5	High
2	Vendor Management	US-03	Assign multiple vendors to an event so that services can be coordinated.	5	High
2	Notifications and Automation	US-04	Send timely event reminders to clients.	3	High
3	Approval Management	US-05	Cancellation requests follow an approval process so that cancellations are controlled.	5	High
3	Venue Management and Apex	US-06	Prevent double bookings so that venue schedules remain accurate.	8	High
4	Security and Access Control	US-07	Role-based access so that users see only appropriate information.	5	High

Sprint	Epic	Story	User story / task	Pts	Priority
4	Reports and Dashboards	US-08	Reports and dashboards so that event operations can be monitored.	5	Medium

4.2 Sprint Schedule

Sprint	Total points	Duration	Start date	End date (planned)
Sprint-1	20	1 day	01 Oct 2026	01 Oct 2026
Sprint-2	20	1 day	02 Oct 2026	02 Oct 2026
Sprint-3	20	1 day	03 Oct 2026	03 Oct 2026
Sprint-4	20	1 day	03 Oct 2026	03 Oct 2026

5. Implementation

5.1 Org Setup

A Developer Edition org was created at developer.salesforce.com/signup, the account was verified by email, and every object, field, flow, class and component described in this document was created through the Setup menu and the Developer Console, working in Lightning Experience.

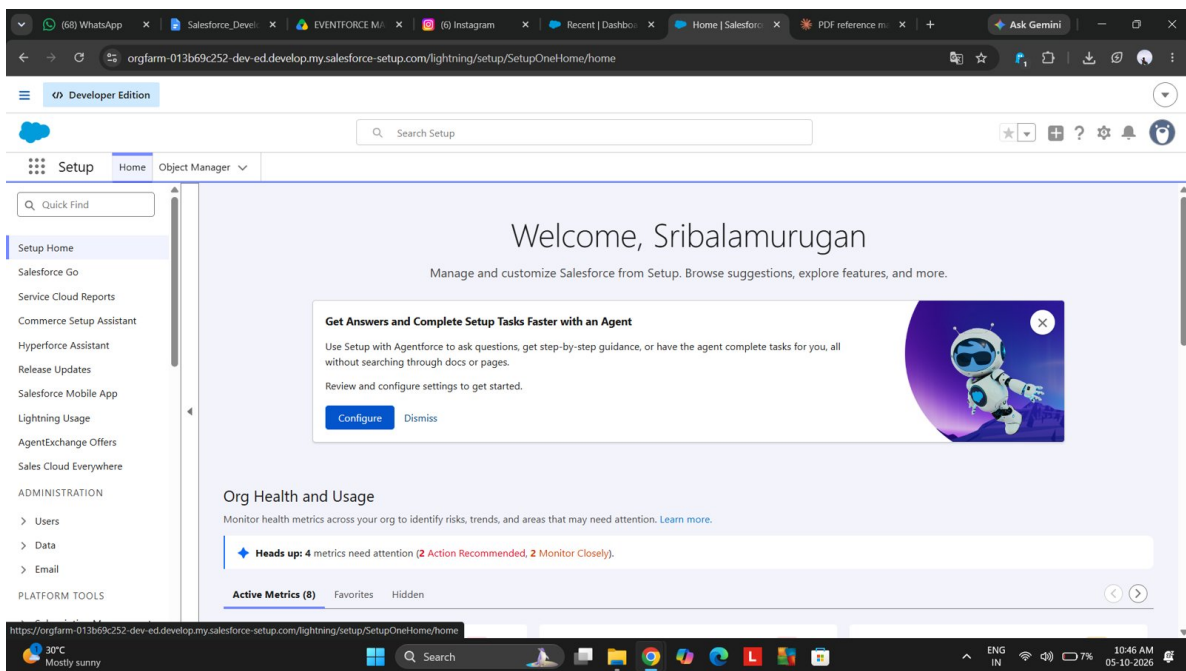


Figure 1: Developer Edition org Setup home page with the logged-in user.

5.2 Data Model

The data model has five custom objects and one junction object. Event, Client, Vendor and Venue use a text record name. Feedback uses an auto-number record name in the format F - {0000}. Every object has Reports and Search enabled and the Notes and Attachments related list added. Event also has Activities and Field History tracking on. A custom tab was created for each object.

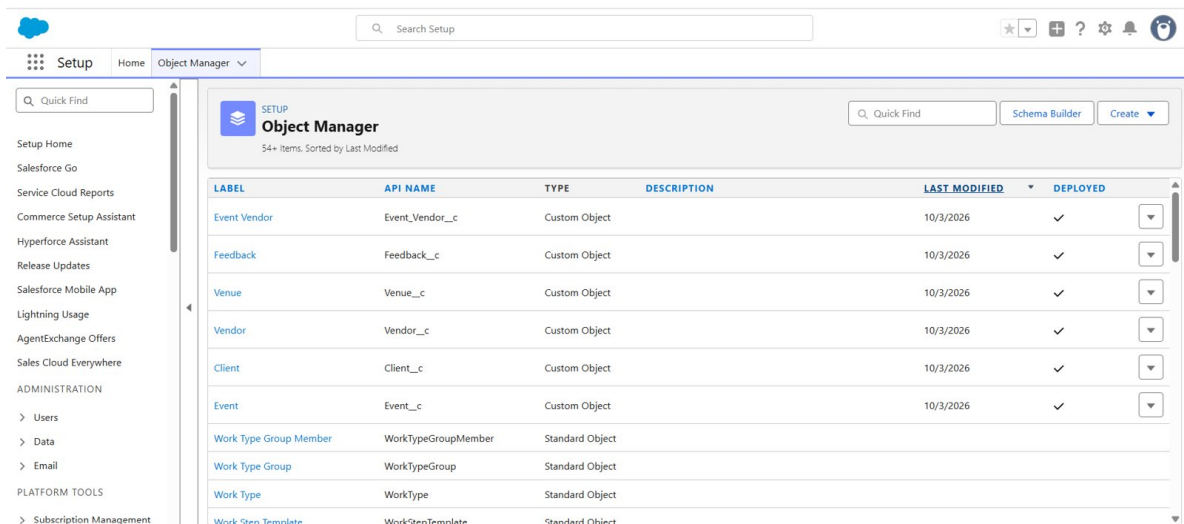


Figure 2: Object Manager showing the custom objects.

Object (API name)	Purpose	Fields we created
Event (Event__c)	One record per event.	Event Name, Event Date (Date), Event Type (picklist), Event Status (picklist), Event Budget (formula, Currency), Client (lookup), Venue (lookup)
Client (Client__c)	One record per client.	Client Name, Email, Phone, Address (Text Area), Country (picklist), City (picklist)
Vendor (Vendor__c)	One record per vendor.	Vendor Name, Email, Phone, Service Type (multi-select picklist), Status (picklist)
Venue (Venue__c)	One record per venue.	Venue Name, Address (Text Area), Location (URL), Capacity (Number), Availability Status (picklist)
Feedback (Feedback__c)	Client feedback for an event.	Feedback ID (auto number), Rating (picklist), Comments (Long Text), Event (lookup), Client (lookup)
EventVendor (EventVendor__c)	Junction object linking events and vendors.	Event (master-detail), Vendor (master-detail)

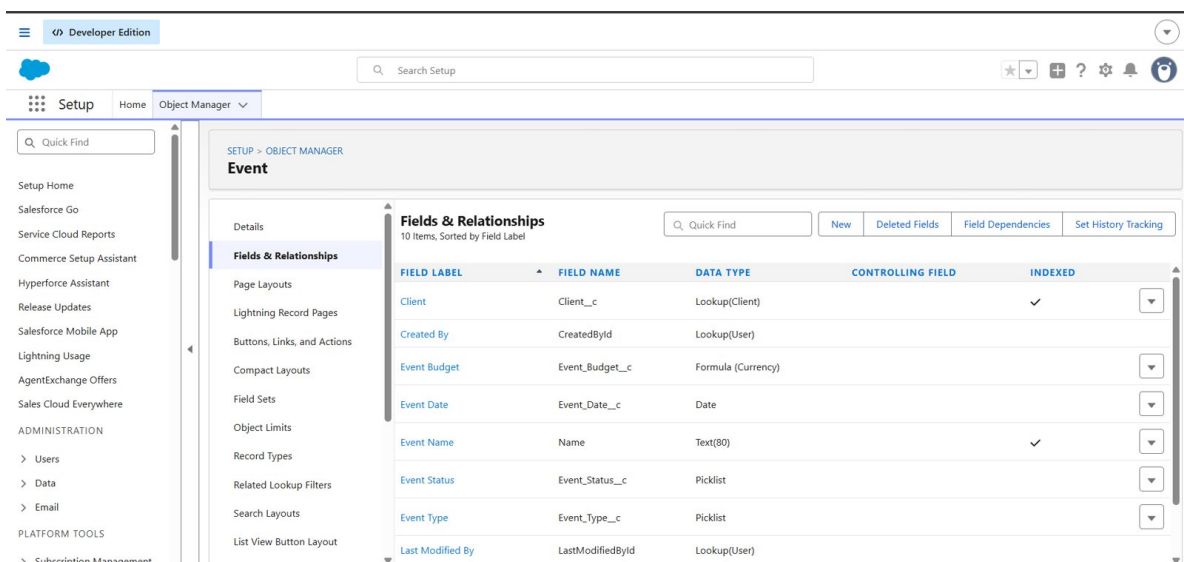


Figure 3: Event object fields and relationships.

The screenshot shows the Salesforce Setup interface in Developer Edition. The left sidebar contains navigation links for Setup, Home, and Object Manager. The main content area is titled 'SETUP > OBJECT MANAGER Client'. Under the 'Fields & Relationships' section, there is a table listing 9 fields for the Client object, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address__c	Text Area(255)		
City	City__c	Picklist		
Client Name	Name	Text(80)		✓
Country	Country__c	Picklist		
Created By	CreatedById	Lookup(User)		
Email	Email__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓

Figure 4: Client object fields.

The screenshot shows the Salesforce Setup interface in Developer Edition. The left sidebar contains navigation links for Setup, Home, and Object Manager. The main content area is titled 'SETUP > OBJECT MANAGER Vendor'. Under the 'Fields & Relationships' section, there is a table listing 8 fields for the Vendor object, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Email	Email__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Phone	Phone__c	Phone		
Service Type	Service_Type__c	Picklist (Multi-Select)		
Status	Status__c	Picklist		
Vendor Name	Name	Text(80)		✓

Figure 5: Vendor object fields.

The screenshot shows the Salesforce Setup interface in Developer Edition. The left sidebar contains navigation links for Setup, Home, and Object Manager. The main content area is titled 'SETUP > OBJECT MANAGER Venue'. Under the 'Fields & Relationships' section, there is a table listing 8 fields for the Venue object, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address__c	Text Area(255)		
Availability Status	Availability_Status__c	Picklist		
Capacity	Capacity__c	Number(18, 0)		
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Location	Location__c	URL(255)		
Owner	OwnerId	Lookup(User,Group)		✓
Venue Name	Name	Text(80)		✓

Figure 6: Venue object fields.

SETUP > OBJECT MANAGER

Feedback

Details

Fields & Relationships
8 Items, Sorted by Field Label

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Client	Client__c	Lookup(Client)		✓
Comments	Comments__c	Long Text Area(32768)		
Created By	CreatedById	Lookup(User)		
Event	Event__c	Lookup(Event)		✓
Feedback	Name	Auto Number		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Rating	Rating__c	Picklist		

Figure 7: Feedback object fields.

Event Planner

Event: **Birthday Party**

Related Details

Event Name	Birthday Party	Owner	Sribalamurugan S
Event Type	Birthday		
Event Date	10/9/2026		
Event Status	Pending Cancellation		
Event Budget	\$10,000.00		
Venue	Open Space at Chennai		
Client	Dharun		
Created By	Sribalamurugan S, 10/3/2026, 9:46 AM	Last Modified By	Sribalamurugan S, 10/3/2026, 10:49 AM

Activity

Filters: All time • All activities • All types

Refresh • Expand All • View All

Upcoming & Overdue

No activities to show.
Get started by sending an email, scheduling a task, and more.

To change what's shown, try changing your filters.

Show All Activities

Figure 8: Sample Event record (Birthday Party) with the Event Budget calculated by the formula field.

Event Planner

Client: **Dharun**

Related Details

Client Name	Dharun	Owner	Sribalamurugan S
Email	sribalamurugan9788@gmail.com		
Phone	(998) 877-6655		
Address			
Country	India		
City	Chennai		
Created By	Sribalamurugan S, 10/3/2026, 9:46 AM	Last Modified By	Sribalamurugan S, 10/3/2026, 9:46 AM

Activity

Filters: All time • All activities • All types

Refresh • Expand All • View All

Upcoming & Overdue

No activities to show.
Get started by sending an email, scheduling a task, and more.

To change what's shown, try changing your filters.

Show All Activities

Figure 9: Sample Client record.

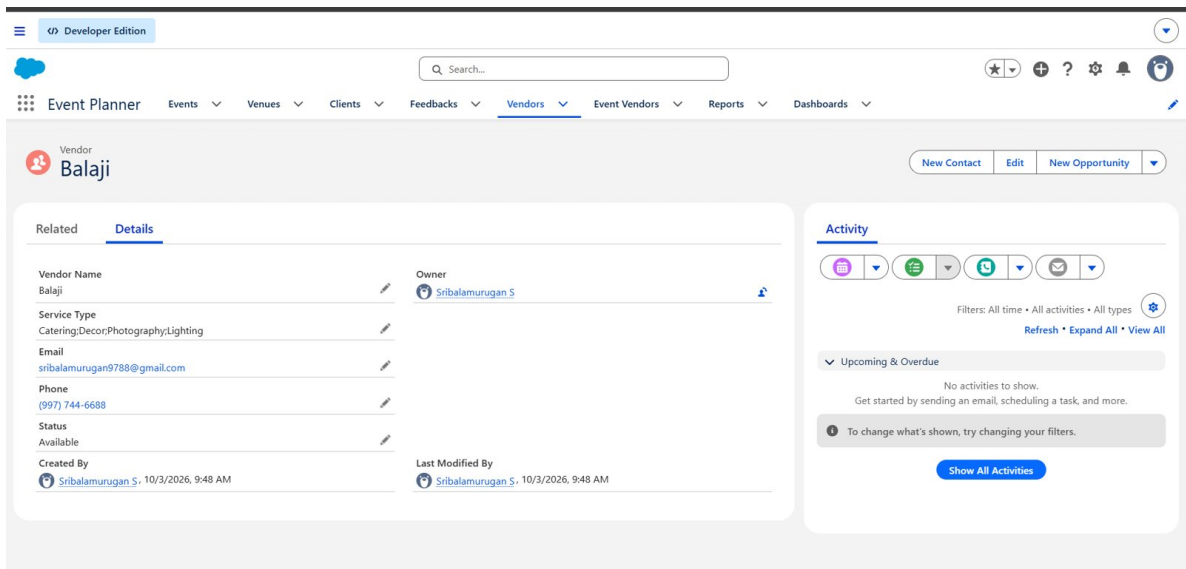


Figure 10: Sample Vendor record.

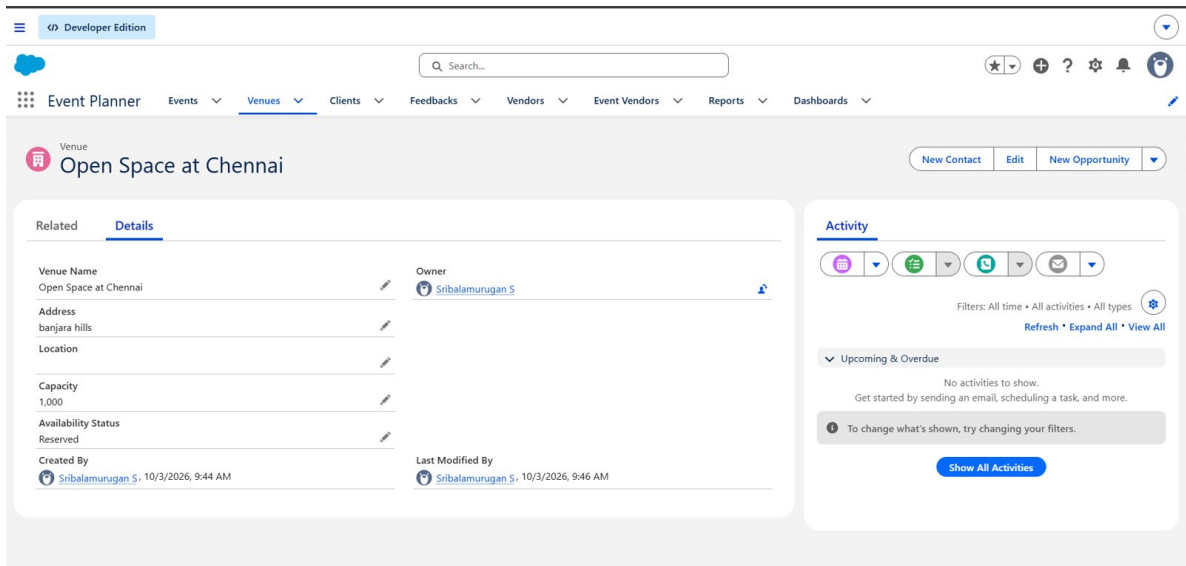


Figure 11: Sample Venue record.

Picklist values

Field	Values
Event Type	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event Status	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Vendor Service Type	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor Status	Available, Booked, Cancelled
Venue Availability Status	Available, Reserved, Not Available
Feedback Rating	1, 2, 3, 4, 5
Client Country and City	Country and city picklists covering India, Philippines, Saudi Arabia, USA, Switzerland and other locations used by the clients

Relationships

Relationship	Type
Event to Client	Lookup

Relationship	Type
Event to Venue	Lookup
Feedback to Event	Lookup
Feedback to Client	Lookup
EventVendor to Event	Master-Detail
EventVendor to Vendor	Master-Detail

Event and Vendor have a many-to-many relationship through the EventVendor junction object.

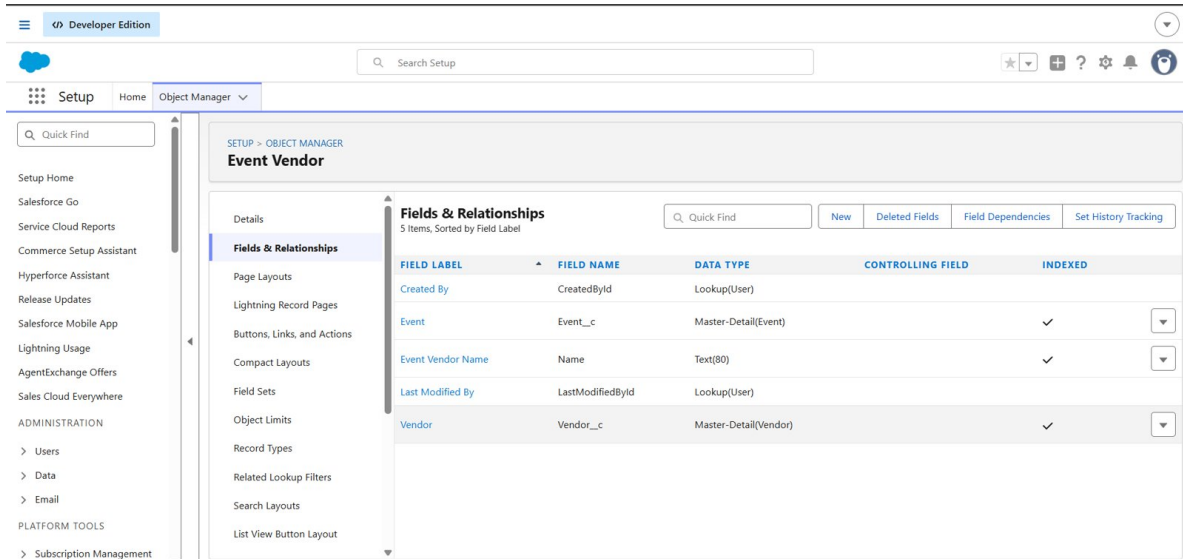


Figure 12: EventVendor object fields with its two master-detail relationships (Event and Vendor).

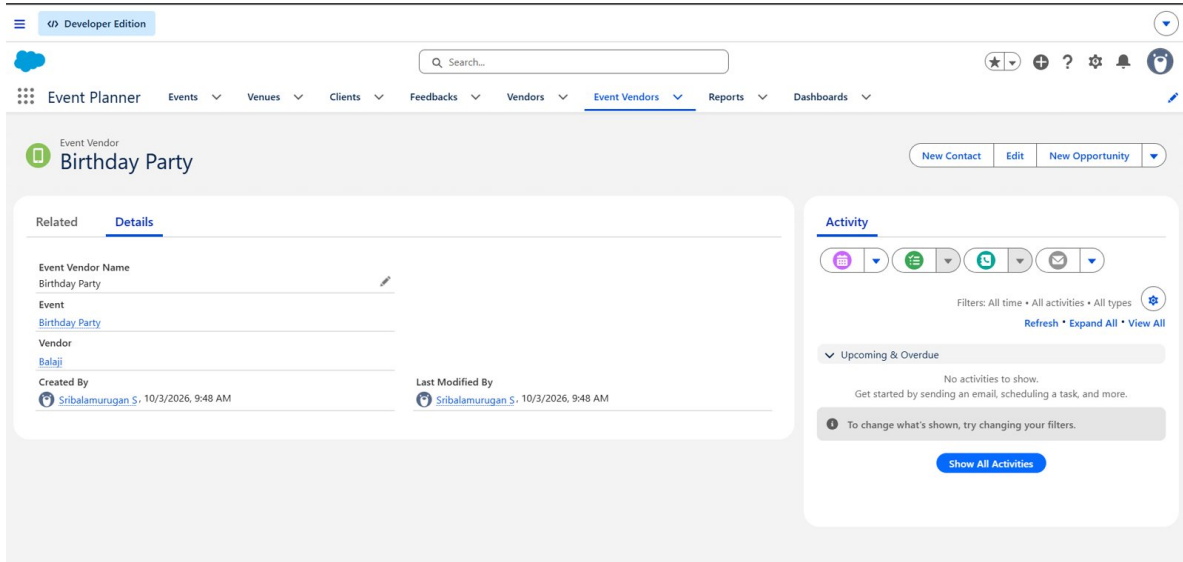


Figure 13: EventVendor record linking the event Birthday Party to the vendor Balaji.

Lookup filter on Feedback. After a client is chosen on a Feedback record, the Event lookup shows only that client's events. The filter on the Event lookup field is: Event: Client equals Feedback: Client, with the filter enabled.

5.3 Validation Rule for Client Email

Property	Value
Rule name	Email_Valid_Address (Active)
Object	Client

Property	Value
Error condition formula	NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\.[a-zA-Z]{2,}\$"))
Error message	Please Enter Valid Email Address
Error location	Email field

5.4 Event Budget Formula Field

Event Budget is a Currency formula field on Event that returns a standard budget for each Event Type.

```
CASE(
  TEXT(Event_Type__c),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)
```

5.5 Client Reminder Flow

A record-triggered Flow named **Client Reminder - 3 Days Before** sends the client a reminder before a confirmed event.

Setting	Value
Object	Event
Trigger	A record is created or updated
Entry condition	Event Status equals Confirmed
Optimized for	Actions and Related Records
Scheduled path	3_Days_Before: time source Event Date, offset 3 days before
Text templates	Subject: Reminder: Your Event {!\$Record.Name} is in 3 days. Body: greets the client by name and gives event name, date, venue and type.
Action	Send Email (Alert_Client_3Day_Reminder): recipient is the Client email, CC is the Event Owner, subject and body from the templates
Status	Saved and activated

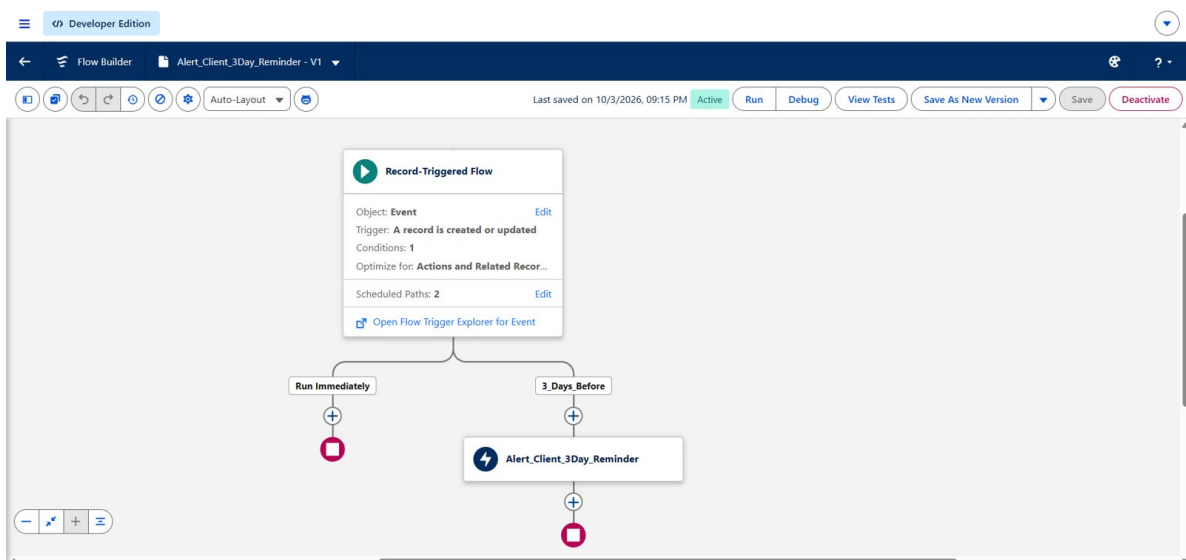


Figure 14: Record-triggered flow on Event (Alert_Client_3Day_Reminder) with the 3_Days_Before scheduled path.

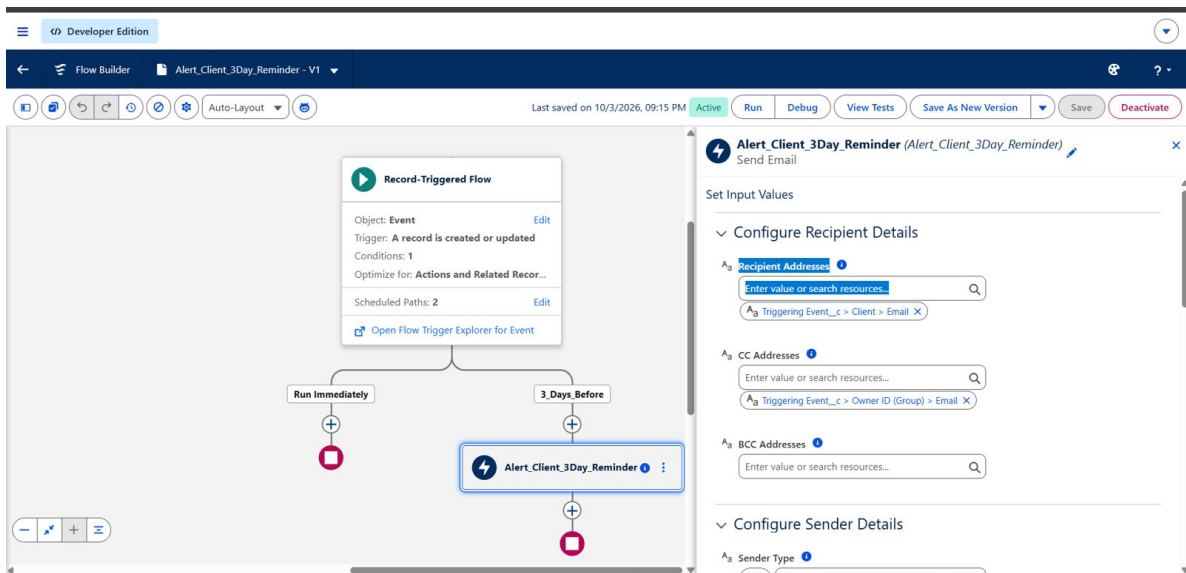


Figure 15: Send Email action with the client as recipient and the event owner in CC.

5.6 Venue Status Update (Apex)

The class **VenueStatusHelper** and the trigger **EventTrigger13** (after insert, after update on Event) keep the venue's availability in step with the event.

Event Status	Effect on the linked venue
Confirmed	Availability Status becomes Reserved
Canceled	Availability Status becomes Available

The source code is in Appendix A.

5.7 Double Booking Prevention (Apex Trigger)

The trigger **PreventDoubleBooking** runs before insert and before update on Event. It collects the venues on the saved events, queries the existing events at those venues, and adds the error *This Venue is already booked on this date*. when another event at the same venue has the same Event Date.

5.8 Nightly Event Completion (Batch and Scheduled Apex)

Component	Role
BatchCompleteEvents	Batch class that selects events with Event Date before today whose status is not Completed and sets the status to Completed.
ScheduleCompleteEvents	Schedulable class that starts the batch with a scope of 200.
Scheduled job	Daily Event Completion, scheduled from Setup (Apex Classes, Schedule Apex), all days of the week, 08:00 PM.

5.9 Approval Process for Event Cancellation

Setting	Value
Process name	Event_Cancellation_Process (Active)
Object	Event
Entry criteria	Event Status equals Pending Cancellation
Approval page fields	Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type, Venue
Initial submitters	Event Owner
Step 1 - Manager Review	All records enter; automatically assigned to a specific approver (Manager)

Setting	Value
Initial submission action	Email alert Alert - Event Cancellation Request to the Event Coordinator role
Final approval actions	Field update Event Status = Canceled; email alert Alert - Cancellation Approved to the Event Owner
Final rejection actions	Email alert Alert - Cancellation Rejected to the Event Owner; field update Event Status = Rejected

Four classic email templates support the process: Event Cancellation Request Notification, Email to Client After Cancellation, Approval Notification to Event Owner, and Rejection Notification to Event Owner.

5.10 Lightning App, Report and Dashboard

Lightning App. The Event Planner app contains the tabs Event, Client, Vendor, Venue, Feedback, Reports and Dashboards and is assigned to the System Administrator profile and the custom profiles.

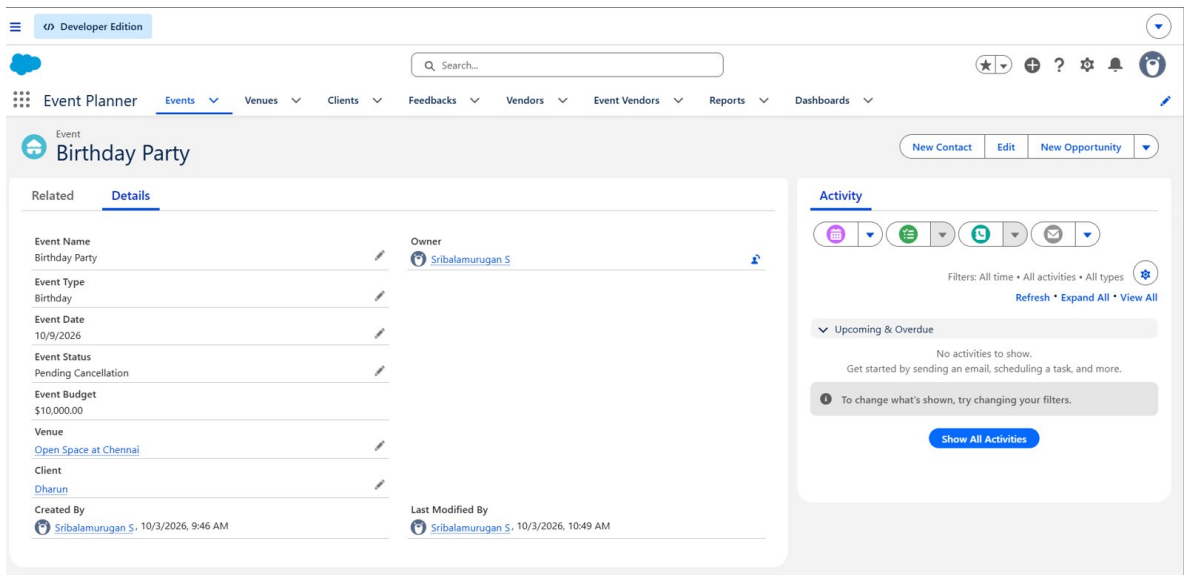


Figure 16: Event Planner Lightning App with its tabs (Events, Venues, Clients, Feedbacks, Vendors, Event Vendors, Reports, Dashboards).

Report. Folder EventForce Report is shared with the Event Coordinator and Vendor Manager roles (view access). The report **Upcoming Events by Month** is built on Events with Client and Venue.

Setting	Value
Columns	Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget
Filters	All Events; Event Date on or after today; Event Status not equal to Canceled
Grouping	Event Date by calendar month
Summaries	Row count and SUM of Event Budget
Chart	Column or line chart

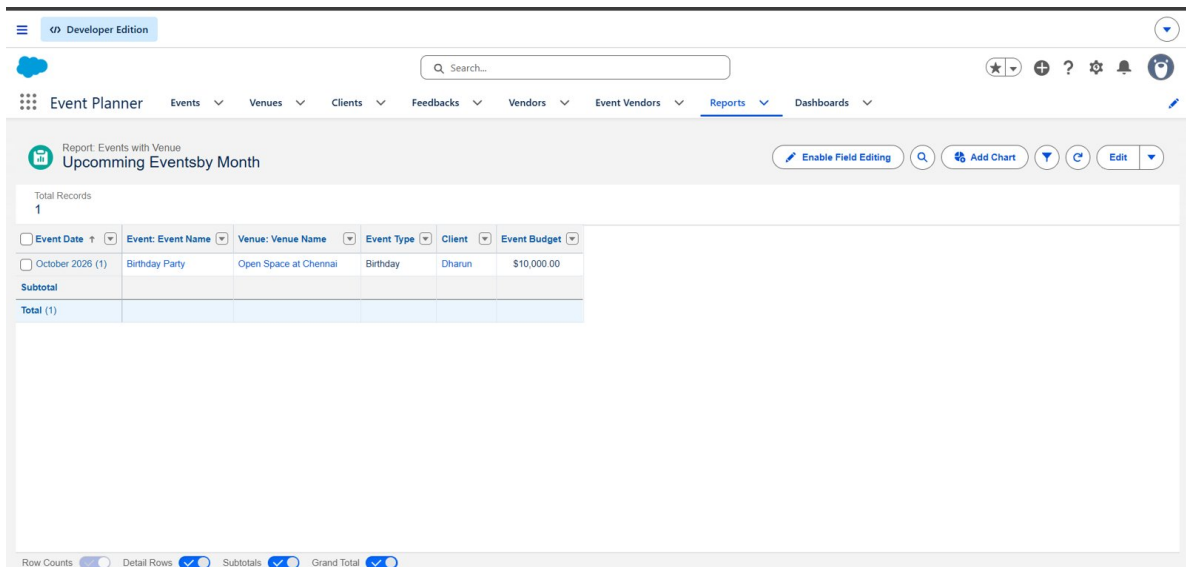


Figure 17: Upcoming Events by Month report.

Dashboard. EventForce Operations Dashboard shows the report as a donut chart (maximum 6 values, subtitle Upcoming Events Overview, light theme).

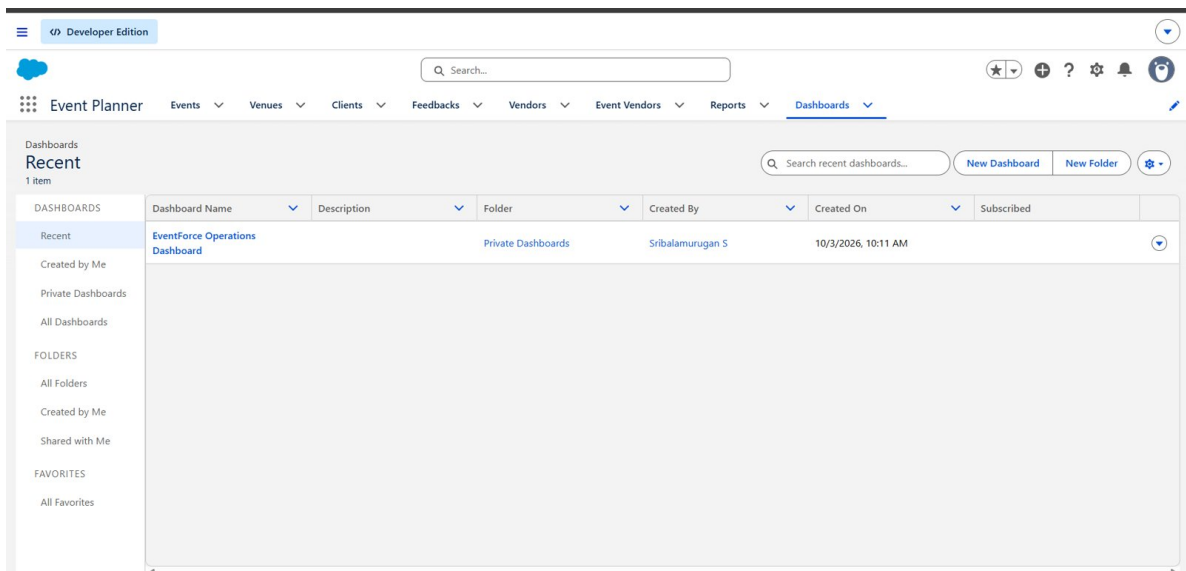


Figure 18: EventForce Operations Dashboard in the Dashboards tab.

5.11 Security

Profiles. Each profile uses a session timeout of 2 hours, passwords that never expire and a minimum password length of 8 (the three profiles cloned from Standard Platform User).

Profile	Based on	Object access
Event Admin	System Administrator	Full access
Event Coordinator	Standard Platform User	Event: Create, Read, Edit, Delete. Client: Read, Edit. Vendor: Read. Venue: Read. Feedback: Create, Read.
Vendor Manager	Standard Platform User	Vendor: Create, Read, Edit, Delete. Event, Venue, Client, Feedback: Read only.
Client Profile	Standard Platform User	Event: Read only (own events). Feedback: Create, Read. Vendor and Venue: no access.

Roles. Event Admin sits under the CEO role. Event Coordinator, Vendor Manager and Client sit under Event Admin.

Users. One user was created per role. The Event Admin user uses the Salesforce license; the other users use the Salesforce Platform license.

Permission Set. Feedback Manager gives Create, Read, Edit and Delete on Feedback and is assigned to the Event Coordinator user.

OWD and sharing rule. The Event object is set to Private for both internal and external access. The sharing rule **Event_Sharing_For_Vendors** shares Event records owned by the Event Coordinator role with the Vendor Manager role at Read Only access.

5.12 Data Migration

Records for Events, Clients, Vendors, Venues and Feedback were loaded with the Data Import Wizard after pre-validation checks to avoid duplicates and keep the data consistent. At least 10 records were created in every object so that the reports show meaningful data.

6. Testing

Each feature is tested from the Lightning interface. The table lists the test cases with the behaviour expected from the design.

No.	Test case	Expected result	Evidence
1	Save a Client with an invalid email	Save is blocked with the email error message	Tested in org
2	Save a Client with a valid email	Client saves	Tested in org
3	Create an Event with Event Type Birthday	Event Budget shows 10,000	See Figure 8
4	Create two events at the same venue on the same date	Second save is blocked with the double-booking error	Tested in org
5	Set an Event to Confirmed with a venue	Venue Availability Status becomes Reserved	Tested in org
6	Assign two vendors to one event through EventVendor	Both vendors appear on the event	Tested in org
7	Confirm an event dated 3 days ahead	Client receives the reminder email, owner in CC	Tested in org
8	Set status to Pending Cancellation and submit; approve	Status becomes Canceled; venue becomes Available; emails sent	Tested in org
9	Submit a cancellation and reject it	Status becomes Rejected; owner is notified	Tested in org
10	Create Feedback and pick a client	Event lookup lists only that client's events	Tested in org
11	Run the scheduled batch with a past-dated event	Event Status becomes Completed	Tested in org
12	Log in as Vendor Manager and Event Coordinator	Each user sees only the access set in the profile, role and sharing rule	Tested in org

7. Challenges Faced and Fixes

7.1 Many events with many vendors

An event can need several vendors and a vendor can serve several events, and Salesforce has no direct many-to-many field. We created the EventVendor junction object with two master-detail relationships, to Event and to Vendor.

7.2 Feedback tied to the right client

Feedback relates to both an event and a client, and a user could pick an event that belongs to someone else. A lookup filter on the Event field limits the choices to events of the selected client.

7.3 Preventing double booking

A venue must not host two events on one date. A validation rule cannot query other records, so we used a before-save Apex trigger that looks up the existing events at the venue and blocks the save with an error.

7.4 Sending a reminder at the right time

The reminder must go out three days before the event, not when the record is saved. We used a scheduled path on the record-triggered Flow with Event Date as the time source and an offset of 3 days before.

7.5 Controlled cancellation

Cancelling an event needs a decision and a record of it. The Pending Cancellation status starts an approval process, and the final actions set the status to Canceled or Rejected and notify the owner.

8. Limitations and Future Scope

Current limitations of the solution:

- The project was built in a Developer Edition org for academic purposes and has not been deployed to a production environment. Real deployment would need sandboxes, Change Sets, Salesforce DX or CI/CD tools.
- Apex test classes have not been written for the triggers, helper class and batch job. They are needed (with at least 75% coverage) before a production deployment.
- The nightly batch marks every past event that is not Completed as Completed, so past Canceled or Rejected events are also changed. The query needs a status filter.
- The double-booking check compares Event Date only and also counts events that are already Canceled.
- The approver in the approval step is a specific user set in the configuration, so it needs maintenance when people change.
- Flows, Apex jobs, approval processes, validation rules and sharing settings need regular monitoring and troubleshooting.

Ideas for future work:

- Deploy the solution from development to a production Salesforce environment with proper deployment tools.
- Extend automation for more event planning and operational activities.
- Add more advanced vendor tracking and performance management.
- Build more reports and dashboards for events, venues, vendors and client feedback.
- Integrate with external systems and services used in real event operations.
- Improve mobile access for event management.
- Scale the solution for larger numbers of events, clients, vendors, venues and users.

9. Conclusion

The EventForce Management System brings client, event, venue, vendor and feedback management, venue booking control, reminders, cancellation approval and reporting into one Salesforce application. The data model with its EventVendor junction object keeps the records connected; the validation rule and formula field protect data quality; Apex prevents double booking and keeps venue availability current; the Flow and the approval process automate communication and decisions; and Profiles, Roles, a Permission Set, OWD and a sharing rule control access. Building it gave our team practical experience with objects, Flows, Apex, Batch and Scheduled Apex, approval processes, reports and security, and the same design can be extended with more automation, integrations and production deployment.

Appendix A: Apex Source Code

VenueStatusHelper

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

EventTrigger13

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(trigger.new);
}
```

PreventDoubleBooking

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                      FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');
```

BatchCompleteEvents

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE Event_Date__c < TODAY AND
            Event_Status__c != \'Completed\''
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
    }
}
```

ScheduleCompleteEvents

```
global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```